

FASTQC

```
#!/bin/bash
#SBATCH --account ibms
#SBATCH --partition=ada
#SBATCH --time=20:00:00
#SBATCH --nodes=1 --ntasks=8
#SBATCH --job-name="FastQC"
#SBATCH --mail-user=[REDACTED]
#SBATCH --mail-type=ALL

# Load FastQC module
module load software/FastQC-0.12.1

# Directories
input_dir="/scratch/[REDACTED]/rawdata/"
output_dir="/scratch/[REDACTED]/fastqc/"

# Process all .fq files
for file_path in "$input_dir"*.fq; do
    if [ -e "$file_path" ]; then
        fastqc "$file_path" -o "$output_dir"
    else
        echo "No .fq files found in $input_dir"
        break
    fi
done
```

rRNA indexing

```
#!/bin/bash
#SBATCH --account ibms
#SBATCH --partition=ada
#SBATCH --time=20:00:00
#SBATCH --nodes=1 --ntasks=8
#SBATCH --job-name="rRNA_index"
#SBATCH --mail-user=[REDACTED]
#SBATCH --mail-type=ALL

# Load module
module load software/bowtie2-2.5.4

# Paths
RRNA_FASTA="/scratch/[REDACTED]/rRNA_reference.fa"
INDEX_PREFIX="/scratch/[REDACTED]/rRNA_index"
THREADS=8

# Build the index
echo "Building Bowtie2 index in large format (.bt2l)..."
bowtie2-build --threads "$THREADS" "$RRNA_FASTA" "$INDEX_PREFIX"

# Confirm output files exist
echo "Checking index files..."
for suffix in 1.bt2l 2.bt2l 3.bt2l 4.bt2l rev.1.bt2l rev.2.bt2l; do
    file="$INDEX_PREFIX}.${suffix}"
    if [[ ! -f "$file" ]]; then
        echo "Missing index file: $file"
        exit 1
    fi
done
```

```
echo "Index build complete"
```

Filtering rRNA reads

```
#!/bin/bash
#SBATCH --account ibms
#SBATCH --partition=ada
#SBATCH --time=5:00:00
#SBATCH --nodes=1 --ntasks=8
#SBATCH --job-name="filter_rRNA"
#SBATCH --mail-user=[REDACTED]
#SBATCH --mail-type=ALL

# Load Bowtie2 module
module load software/bowtie2-2.5.4 || { echo "Error loading Bowtie2 module";
exit 1; }

# Set variables
RAW_DIR="/scratch/[REDACTED]/rawdata"
INDEX_PREFIX="/scratch/[REDACTED]/rRNA_index"
FILTERED_DIR="/scratch/[REDACTED]/filtered"
THREADS=8

# Check Bowtie2 index files exist
for suffix in 1.bt2 2.bt2 3.bt2 4.bt2 rev.1.bt2 rev.2.bt2; do
    if [[ ! -f "${INDEX_PREFIX}.${suffix}" ]]; then
        echo "Missing index file: ${INDEX_PREFIX}.${suffix}"
        exit 1
    fi
done
echo "All Bowtie2 index files found."

# Define sample base names (without _1.fq/_2.fq)
samples=(
[REDACTED]
)

# Loop over samples and filter rRNA
for sample in "${samples[@]"; do
    READ1="${RAW_DIR}/${sample}_1.fq"
    READ2="${RAW_DIR}/${sample}_2.fq"
    OUT1="${FILTERED_DIR}/${sample}_filtered_1.fq"
    OUT2="${FILTERED_DIR}/${sample}_filtered_2.fq"
    LOG="${FILTERED_DIR}/${sample}_bowtie_rRNA.log"

    if [[ -f "$READ1" && -f "$READ2" ]]; then
        echo "Filtering rRNA from sample: $sample"

        bowtie2 -x "$INDEX_PREFIX" \
            -1 "$READ1" -2 "$READ2" \
            --very-sensitive --threads "$THREADS" \
            --un-conc "${FILTERED_DIR}/${sample}_filtered_%.fq" \
            -S /dev/null 2> "$LOG"

        # Confirm output was created
        if [[ -f "$OUT1" && -f "$OUT2" ]]; then
            echo "Finished: $sample"
        else

```

```

        echo "ERROR: Output files missing for $sample"
    fi
else
    echo "Skipping $sample: missing input files"
fi
done

echo "All filtering complete"

```

TRIMMOMATIC

```

#!/bin/bash
#SBATCH --account ibms
#SBATCH --partition=ada
#SBATCH --time=20:00:00
#SBATCH --nodes=1 --ntasks=8
#SBATCH --job-name="Trimmomatic"
#SBATCH --mail-user=[REDACTED]
#SBATCH --mail-type=ALL

# Load Trimmomatic module
module load software/Trimmomatic-0.40-rc1 || { echo "Error loading module"; exit
1; }

# Verify Java is available
java -version || { echo "Java not found"; exit 1; }

# Define paths
adapters="/scratch/[REDACTED]/adapters1.fasta"
input_dir="/scratch/[REDACTED]/rawdata/"
output_dir="/scratch/[REDACTED]/trim/"
trimmomatic_jar="/opt/exp_soft/Trimmomatic-0.40-rc1/trimmomatic-0.40-rc1.jar"

# Check if Trimmomatic JAR exists
if [[ ! -f "$trimmomatic_jar" ]]; then
    echo "Error: Trimmomatic JAR not found at $trimmomatic_jar"
    exit 1
fi

# Define base sample names
samples=(
    [REDACTED]
)

# Loop over samples
for sample in "${samples[@]"; do
    file1="${input_dir}${sample}_1.fq"
    file2="${input_dir}${sample}_2.fq"

    # Ensure both files exist
    if [[ -f "$file1" && -f "$file2" ]]; then
        echo "Processing sample: $sample"

        java -jar "$trimmomatic_jar" \
            PE -phred33 "$file1" "$file2" \
            "${output_dir}${sample}_1_paired.fq" /dev/null \
            "${output_dir}${sample}_2_paired.fq" /dev/null \
            ILLUMINACLIP:"$adapters":2:20:10 \
            LEADING:3 \
            TRAILING:3 \
            SLIDINGWINDOW:4:15 \
            MINLEN:25
    fi
done

```

```
else
    echo "Warning: One or both input files not found for sample $sample,
skipping."
fi
done
```

STAR index generation

```
#!/bin/bash
#SBATCH --account ibms
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1 --ntasks=8
#SBATCH --job-name="STARindex"
#SBATCH --mail-user=[REDACTED]
#SBATCH --mail-type=ALL

# Load STAR module
module load software/STAR-2.7.11b

# Define variables
GENOME_DIR="/scratch/[REDACTED]/STAR_ref_index"
FASTA="/scratch/[REDACTED]/Homo_sapiens.GRCh38.dna_sm.primary_assembly.fa"
GTF="/scratch/[REDACTED]/Homo_sapiens.GRCh38.114.gtf"
SJDB_OVERHANG=100
THREADS=8
LOG="${GENOME_DIR}/star_genome_generate.log"

# Create genome index
STAR \
    --runMode genomeGenerate \
    --genomeDir "$GENOME_DIR" \
    --genomeFastaFiles "$FASTA" \
    --sjdbGTFfile "$GTF" \
    --sjdbOverhang "$SJDB_OVERHANG" \
    --runThreadN "$THREADS" \
    > "$LOG" 2>&1
```

STAR

```
#!/bin/bash
#SBATCH --account ibms
#SBATCH --partition=ada
#SBATCH --time=20:00:00
#SBATCH --nodes=1 --ntasks=8
#SBATCH --job-name="STAR"
#SBATCH --mail-user=[REDACTED]
#SBATCH --mail-type=ALL

# Load modules
module load software/STAR-2.7.11b
module load software/samtools-1.20

# Define paths
INPUT_DIR="/scratch/[REDACTED]/trim/"
OUTPUT_BASE="/scratch/[REDACTED]/STAR_output/"
STAR_INDEX="/scratch/[REDACTED]/STAR_ref_index/"
GTF_FILE="/scratch/[REDACTED]/Homo_sapiens.GRCh38.114.gtf"
FINAL_BAM_DIR="/scratch/[REDACTED]/Aligned/"
```

```

# Set read length
READ_LENGTH=101
SJDB_OVERHANG=$((READ_LENGTH - 1))

# Define sample list
samples=(
)

# Loop through each sample
for sample in "${samples[@]"; do
    FASTQ1="${INPUT_DIR}${sample}_1_paired.fq"
    FASTQ2="${INPUT_DIR}${sample}_2_paired.fq"
    SAMPLE_OUTDIR="${OUTPUT_BASE}${sample}/"

    mkdir -p "$SAMPLE_OUTDIR"
    echo "Processing sample: $sample"

    STAR --runThreadN 8 \
        --genomeDir "$STAR_INDEX" \
        --readFilesIn "$FASTQ1" "$FASTQ2" \
        --outFileNamePrefix "${SAMPLE_OUTDIR}" \
        --outSAMtype BAM SortedByCoordinate \
        --alignEndsType Local \
        --outFilterMultimapNmax 10 \
        --sjdbGTFfile "$GTF_FILE" \
        --sjdbOverhang "$SJDB_OVERHANG" \
        --chimSegmentMin 15 \
        --chimJunctionOverhangMin 10 \
        --outSAMattrRGline ID:$sample \
        --chimOutType SeparateSAMold \
        --twopassMode Basic \
        --outFilterScoreMinOverLread 0.3 \
        --outFilterMatchNminOverLread 0.3 \
        --outFilterMatchNmin 20 \
        --outFilterMismatchNoverReadLmax 0.04 \
        --outReadsUnmapped Fastx

    if [[ $? -ne 0 ]]; then
        echo "STAR alignment failed for $sample. Skipping..."
        continue
    fi

    ## Process aligned BAM
    if [[ -f "${SAMPLE_OUTDIR}Aligned.sortedByCoord.out.bam" ]]; then
        BAM_FILE="${SAMPLE_OUTDIR}${sample}_aligned_reads.bam"
        mv "${SAMPLE_OUTDIR}Aligned.sortedByCoord.out.bam" "$BAM_FILE"

        samtools index "$BAM_FILE"
        samtools flagstat "$BAM_FILE" >
"${SAMPLE_OUTDIR}${sample}_alignment_stats.txt"

        mv "$BAM_FILE" "${FINAL_BAM_DIR}"
        mv "${BAM_FILE}.bai" "${FINAL_BAM_DIR}"
    else
        echo "Aligned.sortedByCoord.out.bam not found for $sample."
    fi

    ## Process chimeric alignments
    if [[ -f "${SAMPLE_OUTDIR}Chimeric.out.sam" ]]; then
        CHIMERIC_BAM_UNSORTED="${SAMPLE_OUTDIR}${sample}_chimeric_reads.unsorted.bam"
        CHIMERIC_BAM="${SAMPLE_OUTDIR}${sample}_chimeric_reads.bam"

```

```

        samtools view -Sb "${SAMPLE_OUTDIR}Chimeric.out.sam" >
"$CHIMERIC_BAM_UNSORTED"
        samtools sort -o "$CHIMERIC_BAM" "$CHIMERIC_BAM_UNSORTED"
        rm "$CHIMERIC_BAM_UNSORTED"

        samtools index "$CHIMERIC_BAM"
        samtools flagstat "$CHIMERIC_BAM" >
"${SAMPLE_OUTDIR}${sample}_chimeric_stats.txt"
    else
        echo "Chimeric.out.sam not found for $sample."
    fi

    echo "Finished processing $sample"
done

echo "All samples processed."

```

FEATURECOUNTS

```

#!/bin/bash
#SBATCH --account ibms
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1 --ntasks=4
#SBATCH --job-name="FeatureCounts"
#SBATCH --mail-user=[REDACTED]
#SBATCH --mail-type=ALL

set -e
set -x

# Load necessary module
module load software/subread-2.0.6 || { echo "Failed to load subread module";
exit 1; }

# Define GTF file and directories
GTF_FILE="/scratch/[REDACTED]/Homo_sapiens.GRCh38.114.gtf"
OUTPUT_DIR="/scratch/[REDACTED]/ALL"
INPUT_DIR="/scratch/[REDACTED]/Aligned"

# List of BAM files to process
BAM_FILES=(
    "[REDACTED]_aligned_reads.bam"
    "[REDACTED]_aligned_reads.bam"
    "[REDACTED]_aligned_reads.bam"
    "[REDACTED]_aligned_reads.bam"
    "[REDACTED]_aligned_reads.bam"
    "[REDACTED]_aligned_reads.bam"
    "[REDACTED]_aligned_reads.bam"
)

BAM_PATHS=()
for BAM_FILE_NAME in "${BAM_FILES[@]}; do
    BAM_PATH="$INPUT_DIR/$BAM_FILE_NAME"
    if [ -f "$BAM_PATH" ]; then
        BAM_PATHS+=("$BAM_PATH")
    else
        echo "Warning: BAM file not found: $BAM_PATH" >&2
    fi
done

# check BAM files to process

```

```

if [ ${#BAM_PATHS[@]} -eq 0 ]; then
    echo "No BAM files found. Exiting." >&2
    exit 1
fi

# Set output and log file paths
OUTPUT_FILE="$OUTPUT_DIR/featureCounts_merged.txt"
LOG_FILE="$OUTPUT_DIR/featureCounts_merged.log"

# Run featureCounts on all BAM files
echo "Running featureCounts on ${#BAM_PATHS[@]} BAM files..."
featureCounts \
    -a "$GTF_FILE" \
    -o "$OUTPUT_FILE" \
    -p \
    -T 4 \
    -t exon \
    -g gene_id \
    --countReadPairs \
    "${BAM_PATHS[@]}" > "$LOG_FILE" 2>&1

if [ $? -eq 0 ]; then
    echo "FeatureCounts completed successfully. Output: $OUTPUT_FILE"
else
    echo "FeatureCounts encountered an error. See log: $LOG_FILE" >&2
    exit 1
fi

# Post-process header in featureCounts output to clean sample names
TMP_FILE="${OUTPUT_FILE%.txt}_cleaned.txt"

awk 'NR==1 { print; next }
     NR==2 {
         printf $1
         for (i = 2; i <= NF; i++) {
             gsub(".*/", "", $i)
             gsub("_aligned_reads.bam", "", $i)
             printf "\t%s", $i
         }
         printf "\n"
         next
     }
     { print }' "$OUTPUT_FILE" > "$TMP_FILE"

mv "$TMP_FILE" "$OUTPUT_FILE"
echo "Cleaned sample names in: $OUTPUT_FILE"

echo "All FeatureCounts processing complete."

```

FORMATTING COUNT MATRIX

```

#!/bin/bash
#SBATCH --account ibms
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1 --ntasks=4
#SBATCH --job-name="Feature format"
#SBATCH --mail-user=[REDACTED]
#SBATCH --mail-type=ALL

# Load Conda module

```

```
module load python/miniconda3-py3.9
```

```
# Initialize Conda
source "$(conda info --base)/etc/profile.d/conda.sh"
```

```
# Activate Conda environment
conda activate rnaseq-norm-env
```

```
# Run script
python /scratch/[REDACTED]/Feature_process.py
```

```
.....

import os
import pandas as pd
from mygene import MyGeneInfo

# Define file paths
base_dir = "/scratch/[REDACTED]/ALL"
input_file = os.path.join(base_dir, "featureCounts_merged.txt")
noheader_file = os.path.join(base_dir, "noheader_featureCounts_merged.txt")
symbol_file = os.path.join(base_dir, "featureCounts_symbol.txt")
final_output_file = os.path.join(base_dir, "featureCount_CM.txt")
unmapped_log_file = os.path.join(base_dir, "unmapped_ensembl_ids.txt")

# Remove header
print("Step 1: Removing header...")

if not os.path.isfile(input_file):
    raise FileNotFoundError(f"Error: {input_file} not found.")

with open(input_file, "r") as infile, open(noheader_file, "w") as outfile:
    next(infile) # Skip the first line
    for line in infile:
        outfile.write(line)

print(f"Header removed. Saved to: {noheader_file}")

# Map ENSEMBL IDs to gene symbols
print("Step 2: Mapping ENSEMBL IDs to gene symbols...")

df = pd.read_csv(noheader_file, sep="\t")
mg = MyGeneInfo()
ensembl_ids = df['Geneid'].tolist()

results = mg.querymany(ensembl_ids, scopes='ensembl.gene', fields='symbol',
species='human')

id_to_symbol = {}
unmapped_ids = []

for item in results:
    if 'symbol' in item and not item.get('notfound', False):
        id_to_symbol[item['query']] = item['symbol']
    else:
        unmapped_ids.append(item['query'])

df_filtered = df[df['Geneid'].isin(id_to_symbol)].copy()
df_filtered['Geneid'] = df_filtered['Geneid'].map(id_to_symbol)

df_filtered.to_csv(symbol_file, sep="\t", index=False)
print(f"Gene symbol matrix saved to: {symbol_file}")

with open(unmapped_log_file, "w") as f:
```

```

        f.write("Unmapped_ENS_IDS\n")
        for eid in unmapped_ids:
            f.write(eid + "\n")

print(f"Unmapped ENSEMBL IDs saved to: {unmapped_log_file}")

# Drop columns 2-6
print("Step 3: Dropping columns 2-6...")

df_symbol = pd.read_csv(symbol_file, sep='\t', comment='#', low_memory=False)

if df_symbol.shape[1] < 6:
    raise ValueError("Not enough columns to drop indices 1 to 5.")

cols_to_drop = df_symbol.columns[1:6]
df_final = df_symbol.drop(columns=cols_to_drop)

df_final.to_csv(final_output_file, sep='\t', index=False)
print(f"Final filtered file saved to: {final_output_file}")

```

PCA

```

#!/usr/bin/env Rscript

suppressPackageStartupMessages({
  library(edgeR); library(limma); library(ggplot2); library(ggrepel)
})

# Files
out_dir <- "/scratch/[REDACTED]/DGE/"
meta_file <- "/scratch/[REDACTED]sample_id2.tsv"
count_file <- "/scratch/[REDACTED]/featureCount_CM.txt"
dir.create(out_dir, recursive=TRUE, showWarnings=FALSE)

# Metadata
meta <- read.delim(meta_file, stringsAsFactors=FALSE, check.names=FALSE)
colnames(meta) <- trimws(colnames(meta))
meta$SAMPLE <- trimws(meta$SAMPLE)
meta$CONDITION <- trimws(meta$CONDITION)
meta$MYC <- trimws(meta$MYC)
meta <- meta[meta$CONDITION %in% c("GCB","NGCB"),]

# Counts and sample matching
counts <- read.delim(count_file, stringsAsFactors=FALSE, check.names=FALSE)
rownames(counts) <- make.unique(as.character(counts$Geneid))
counts <- counts[,-1,drop=FALSE]
colnames(counts) <- trimws(colnames(counts))
common <- intersect(meta$SAMPLE,colnames(counts))
if(!length(common)) stop("No matching samples found.")
counts <- counts[,common,drop=FALSE]
meta <- meta[match(common,meta$SAMPLE),,drop=FALSE]
if(!all(meta$SAMPLE==colnames(counts))) stop("Sample mismatch.")

# Filtering and normalisation
dge <- DGEList(counts=counts)
keep <- rowSums(cpm(dge)>1) >= ceiling(ncol(dge)*0.50)
dge <- dge[keep,,keep.lib.sizes=FALSE]
dge <- calcNormFactors(dge,method="TMM")

# Voom transformation
design <- matrix(1,nrow=ncol(dge),ncol=1)
colnames(design) <- "Intercept"
v <- voom(dge,design,plot=FALSE)

```

```

expr <- v$E

# Top 500 variable genes
gene_variance <- apply(expr,1,var)
gene_variance <- gene_variance[is.finite(gene_variance)]
top_n <- min(500,length(gene_variance))
genes <- names(sort(gene_variance,decreasing=TRUE))[1:top_n]
expr_pca <- expr[genes,,drop=FALSE]

# PCA
pca <- prcomp(t(expr_pca),center=TRUE,scale.=TRUE)
variance <- round(100*pca$sdev^2/sum(pca$sdev^2),1)

pca_df <- data.frame(
  Sample=colnames(expr_pca),
  PC1=pca$x[,1],
  PC2=pca$x[,2],
  CONDITION=meta$CONDITION,
  MYC=meta$MYC
)

# Plot
coo_colours <- c(GCB="#F8766D",NGCB="#00BFC4")
myc_shapes <- c(NR=16,R=17)

p <- ggplot(pca_df,aes(PC1,PC2))+
  geom_point(aes(colour=CONDITION,shape=MYC),size=3.5)+
  geom_text_repel(aes(label=Sample,colour=CONDITION),size=3.2,
    max.overlaps=Inf,box.padding=.30,point.padding=.20,
    force=1,min.segment.length=Inf,show.legend=FALSE)+
  scale_colour_manual(name="COO",values=coo_colours)+
  scale_shape_manual(name="MYC FISH",values=myc_shapes)+
  labs(title="Unsupervised PCA",
    x=paste0("PC1 (",variance[1],"% variance)"),
    y=paste0("PC2 (",variance[2],"% variance)"))+
  theme_bw(base_size=14)+
  theme(plot.title=element_text(hjust=.5,face="bold",size=16),
    axis.title=element_text(size=13),
    axis.text=element_text(size=11,colour="black"),
    panel.border=element_rect(colour="black",fill=NA,linewidth=.8),
    legend.title=element_text(face="bold",size=12),
    legend.text=element_text(size=11),
    legend.position="right")

print(p)

# Outputs
ggsave(file.path(out_dir,"Unsupervised_PCA_GCB_NGCB.pdf"),p,
  width=8,height=6.5)
ggsave(file.path(out_dir,"Unsupervised_PCA_GCB_NGCB.png"),p,
  width=8,height=6.5,dpi=300)

write.csv(pca_df,
  file.path(out_dir,"Unsupervised_PCA_coordinates.csv"),
  row.names=FALSE)

write.csv(
  data.frame(Gene=genes,Variance=gene_variance[genes]),
  file.path(out_dir,"Top500_variable_genes_unsupervised_PCA.csv"),
  row.names=FALSE
)

```

sample_id2.tsv

SAMPLE	CONDITION	MYC
17326	GCB	R
13437	GCB	R
5646	GCB	R
18681	GCB	R
17565	GCB	R
9349	NGCB	R
17845	GCB	NR
22461	GCB	NR
10164	NGCB	NR
15390	NGCB	NR
3286	GCB	NR
18258	NGCB	NR
2268	GCB	NR
13904	NGCB	NR
13193	GCB	NR
18489	GCB	NR
19517	GCB	NR
25395	NGCB	NR
14716	NGCB	NR
29665	NGCB	NR
22392	NGCB	NR

Differential gene expression

```
#!/bin/bash
#SBATCH --account=ibms
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --job-name="DGE"
#SBATCH --mail-user=[REDACTED]
#SBATCH --mail-type=ALL

# Load Conda module
module load python/miniconda3-py3.9

# Initialize Conda
source "$(conda info --base)/etc/profile.d/conda.sh"

# Activate environment
conda activate rnaseq-norm-env

Rscript -e 'if (!require("FactoMineR")) BiocManager::install("FactoMineR") '
Rscript -e 'if (!require("factoextra")) BiocManager::install("factoextra") '

# Run R script
Rscript /scratch/[REDACTED]/DGE_analysis.r
```

DGE_analysis.r

```
#!/usr/bin/env Rscript

library(DESeq2); library(ggplot2); library(dplyr); library(tibble)
library(pheatmap); library(edgeR); library(ggrepel); library(matrixStats)
```

```

confirm <- function(x) cat(paste0(x, " ... done\n"))

sample_file <- "/scratch/[REDACTED]/sample_id.tsv"
count_file <- "/scratch/[REDACTED]/featureCount_CM.txt"
output_dir <- "/scratch/[REDACTED]/DGE"
if(!dir.exists(output_dir)) dir.create(output_dir,recursive=TRUE)

samples <- read.table(sample_file,header=TRUE,sep="\t",stringsAsFactors=FALSE)
samples$SAMPLE <- as.character(samples$SAMPLE)
counts <-
read.table(count_file,header=TRUE,sep="\t",stringsAsFactors=FALSE,check.names=FALSE)

counts <- counts %>% group_by(Geneid) %>%
  summarise(across(where(is.numeric),sum),.groups="drop") %>% as.data.frame()
rownames(counts) <- counts$Geneid; counts <- counts[,-1]

common <- intersect(colnames(counts),samples$SAMPLE)
counts <- counts[,common]
samples <- samples[samples$SAMPLE %in% common,]
samples <- samples[match(common,samples$SAMPLE),]
samples <- samples %>% filter(CONDITION %in% c("R","NR"))
counts <- counts[,samples$SAMPLE]

dge <- DGEList(counts=counts)
keep <- rowSums(cpm(dge)>1) >= ceiling(ncol(counts)*.5)
counts <- counts[keep,,drop=FALSE]

samples$CONDITION <- factor(samples$CONDITION,levels=c("NR","R"))
if(any(is.na(samples$CONDITION))) stop("Missing CONDITION values")

dds <- DESeqDataSetFromMatrix(countData=counts,colData=samples,
                             design=~CONDITION)

dds <- DESeq(dds)
vsd <- vst(dds,blind=FALSE)

# PCA
rv <- rowVars(assay(vsd))
sel <- order(rv,decreasing=TRUE)[seq_len(min(1000,length(rv)))]
pca <- prcomp(t(assay(vsd)[sel,]),scale.=FALSE)
percentVar <- round(100*pca$sdev^2/sum(pca$sdev^2))
pcaData <- data.frame(PC1=pca$x[,1],PC2=pca$x[,2],
                     CONDITION=samples$CONDITION,name=rownames(pca$x))

p <- ggplot(pcaData,aes(PC1,PC2,color=CONDITION))+
  geom_point(size=3,alpha=.85)+
  geom_text_repel(aes(label=name),size=3,max.overlaps=50)+
  labs(x=paste0("PC1  (",percentVar[1],"%)" ),
       y=paste0("PC2  (",percentVar[2],"%)" ),
       title="PCA",color="MYC FISH")+
  theme_bw(base_size=14)+
  theme(plot.title=element_text(hjust=.5,face="bold"),
        legend.title=element_text(face="bold"),
        panel.grid.minor=element_blank())

write.csv(pcaData,file.path(output_dir,"PCA_data.csv"),row.names=FALSE)
ggsave(file.path(output_dir,"PCA_plot.png"),p,width=8,height=6,dpi=300)

# Differential expression
res <- results(dds,contrast=c("CONDITION","R","NR"))
res <- lfcShrink(dds,coef="CONDITION_R_vs_NR",res=res,type="ashr")
res_df <- as.data.frame(res); res_df$GeneID <- rownames(res_df)
write.csv(res_df,file.path(output_dir,"DGE_results_R_vs_NR.csv"),row.names=FALSE)
)

```

```

sig_genes <- subset(res_df, padj < .05)
write.csv(sig_genes, file.path(output_dir, "DGE_significant_genes_R_vs_NR.csv"), row.names=FALSE)

overexpressed <- sig_genes[sig_genes$log2FoldChange >= 1, ]
overexpressed <- overexpressed[order(overexpressed$padj), ]
write.csv(overexpressed, file.path(output_dir, "DGE_overexpressed_genes_R_vs_NR.csv"), row.names=FALSE)

underexpressed <- sig_genes[sig_genes$log2FoldChange <= -1, ]
underexpressed <- underexpressed[order(underexpressed$padj), ]
write.csv(underexpressed, file.path(output_dir, "DGE_underexpressed_genes_R_vs_NR.csv"), row.names=FALSE)

write.csv(counts(dds, normalized=TRUE),
          file.path(output_dir, "Normalized_counts.csv"), row.names=TRUE)
write.csv(assay(vsd), file.path(output_dir, "VST_counts.csv"), row.names=TRUE)

pdf(file.path(output_dir, "Dispersion_plot.pdf"))
plotDispEsts(dds); dev.off()

sink(file.path(output_dir, "DGE_summary.txt")); summary(res); sink()

# MA plot
ma_data <- as.data.frame(res) %>%
  filter(!is.na(baseMean), !is.na(log2FoldChange)) %>%
  mutate(sig=case_when(
    padj < .05 & log2FoldChange >= 1 ~ "Upregulated",
    padj < .05 & log2FoldChange <= -1 ~ "Downregulated",
    TRUE ~ "Not Significant"))

ma_plot <- ggplot(ma_data, aes(log10(baseMean+1), log2FoldChange, colour=sig)) +
  geom_point(size=1.5, alpha=.6) +
  geom_hline(yintercept=0, linetype="dashed") +
  scale_colour_manual(values=c(Upregulated="red", Downregulated="blue",
                                `Not Significant`="grey")) +
  labs(title="MA Plot: R vs NR", x="Mean Expression (log10 baseMean)",
        y="Log2 Fold Change") +
  theme_minimal()

ggsave(file.path(output_dir, "MA_plot.pdf"), ma_plot, width=8, height=6)

# Volcano plot
volcano_data <- res_df %>%
  filter(!is.na(log2FoldChange), !is.na(pvalue), pvalue > 0) %>%
  mutate(negLog10P = -log10(pvalue),
         sig=case_when(
           padj < .05 & log2FoldChange >= 1 ~ "Upregulated",
           padj < .05 & log2FoldChange <= -1 ~ "Downregulated",
           TRUE ~ "Not Significant"))

top_genes <- bind_rows(
  volcano_data %>% filter(padj < .05, log2FoldChange >= 1) %>% arrange(padj) %>%
  slice_head(n=5),
  volcano_data %>% filter(padj < .05, log2FoldChange <= -1) %>% arrange(padj) %>%
  slice_head(n=5))

volcano <- ggplot(volcano_data, aes(log2FoldChange, negLog10P, colour=sig)) +
  geom_point(alpha=.6, size=1.5) +
  geom_text_repel(data=top_genes, aes(label=GeneID), size=3.5,
                  fontface="bold", show.legend=FALSE) +
  geom_vline(xintercept=c(-1, 1), linetype="dashed") +
  geom_hline(yintercept=-log10(.05), linetype="dashed") +

```

```

scale_colour_manual(values=c(Upregulated="red",Downregulated="blue",
                             `Not Significant`="grey"))+
labs(title="Volcano Plot: R vs NR",x="Log2 Fold Change",
      y="-Log10 P-value",colour="Significance")+
theme_minimal()

ggsave(file.path(output_dir,"Volcano_plot_Top5_Up_Down.pdf"),
        volcano,width=9,height=7)

# Heatmap: top 50 significant genes
annotation_col <-
data.frame(MYC=samples$CONDITION,SEX=samples$SEX,HIV=samples$HIV)
rownames(annotation_col) <- samples$SAMPLE

top_genes <- sig_genes %>% arrange(padj) %>% slice_head(n=50) %>% pull(GeneID)
mat <- assay(vsd)[top_genes,,drop=FALSE]
mat_scaled <- t(scale(t(mat))); mat_scaled[is.na(mat_scaled)] <- 0

pdf(file.path(output_dir,"Heatmap_top50_significant_genes.pdf"),width=8,height=8)
)
pheatmap(mat_scaled,annotation_col=annotation_col,show_rownames=TRUE,
         fontsize_row=7,cluster_rows=TRUE,cluster_cols=TRUE,
         main="Top 50 Significant Genes")
dev.off()

```

sample id.tsv

SAMPLE	CONDITION	SEX	HIV
2268	NR	M	N
3286	NR	F	N
19517	NR	M	N
13193	NR	F	N
18489	NR	F	N
13437	R	M	P
5646	R	M	P
17326	R	M	P
17845	NR	M	N
22461	NR	M	P
18681	R	F	P
17565	R	M	N

```

#!/bin/bash
#SBATCH --account=ibms
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --job-name="GO"
#SBATCH --mail-user=[REDACTED]
#SBATCH --mail-type=ALL

#Load Conda module
module load python/miniconda3-py3.9

#Initialize Conda
source "$(conda info --base)/etc/profile.d/conda.sh"

#Activate environment
conda activate rnaseq-norm-env

#Run script
Rscript /scratch/[REDACTED]/GO_analysis.r

```

GO analysis.r

```
#!/usr/bin/env Rscript

library(clusterProfiler); library(enrichplot); library(org.Hs.eg.db)
library(dplyr); library(readr); library(ggplot2); library(patchwork)

output_dir <- "/scratch/████████/GO/"
if(!dir.exists(output_dir)) dir.create(output_dir,recursive=TRUE)

pkgs <-
c("clusterProfiler","enrichplot","org.Hs.eg.db","dplyr","readr","ggplot2","patch
work")
for(p in pkgs) if(!requireNamespace(p,quietly=TRUE)){
  if(p %in% c("clusterProfiler","enrichplot","org.Hs.eg.db"))
    BiocManager::install(p,ask=FALSE,update=FALSE)
  else install.packages(p)
}
invisible(lapply(pkgs,library,character.only=TRUE))

input_file <- "/scratch/████████/DEGE/DEGE_results_R_vs_NR.csv"
if(!file.exists(input_file)) stop("Input file does not exist: ",input_file)

de_res <- read_csv(input_file,show_col_types=FALSE)
req <- c("GeneID","log2FoldChange","padj")
miss <- setdiff(req,colnames(de_res))
if(length(miss)>0) stop("Missing columns: ",paste(miss,collapse=" "))

de_res <- de_res %>% filter(!is.na(GeneID),!is.na(log2FoldChange),!is.na(padj))

id_map <- bitr(unique(de_res$GeneID),fromType="SYMBOL",toType="ENTREZID",
  OrgDb=org.Hs.eg.db) %>% distinct(SYMBOL,ENTREZID)

de_res <- de_res %>% left_join(id_map,by=c("GeneID"="SYMBOL")) %>%
  filter(!is.na(ENTREZID)) %>% arrange(padj) %>%
  distinct(ENTREZID,.keep_all=TRUE)

sig_genes <- de_res %>% filter(padj<.05,abs(log2FoldChange)>=1)
up_genes <- sig_genes %>% filter(log2FoldChange>0) %>% pull(ENTREZID) %>%
  unique()
down_genes <- sig_genes %>% filter(log2FoldChange<0) %>% pull(ENTREZID) %>%
  unique()
universe <- unique(de_res$ENTREZID)

cat("\nGO ANALYSIS SUMMARY\n",
  "Total genes tested: ",length(universe)," \n",
  "Significant genes: ",length(unique(sig_genes$ENTREZID))," \n",
  "Upregulated genes: ",length(up_genes)," \n",
  "Downregulated genes: ",length(down_genes)," \n",sep="")

run_go <- function(genes,universe,ontology,prefix){
  cat("\nRunning GO:",prefix,ontology,"| Genes:",length(genes)," \n")
  if(length(genes)<3) return(NULL)

  ego <- tryCatch(
    enrichGO(gene=genes,universe=universe,OrgDb=org.Hs.eg.db,
      keyType="ENTREZID",ont=ontology,pvalueCutoff=.05,
      pAdjustMethod="BH",qvalueCutoff=.05,minGSSize=10,
      maxGSSize=500,readable=TRUE),
    error=function(e){message("enrichGO failed: ",e$message);NULL})

  if(is.null(ego)) return(NULL)
```

```

df <- as.data.frame(ego)
write.csv(df, file.path(output_dir, paste0(prefix, "_GO_", ontology, ".csv")),
          row.names=FALSE)
if(nrow(df)==0) return(ego)

n <- min(20, nrow(df))

pdf(file.path(output_dir, paste0(prefix, "_GO_", ontology, "_dotplot.pdf")),
    width=12, height=8)
print(dotplot(ego, showCategory=n, title=paste0(prefix, " GO ", ontology)))
dev.off()

pdf(file.path(output_dir, paste0(prefix, "_GO_", ontology, "_barplot.pdf")),
    width=12, height=8)
print(barplot(ego, showCategory=n, title=paste0(prefix, " GO ", ontology)))
dev.off()

if(nrow(df)>=2) {
  tryCatch({
    pdf(file.path(output_dir, paste0(prefix, "_GO_", ontology, "_emapplot.pdf")),
        width=12, height=8)
    print(emapplot(pairwise_termsim(ego), showCategory=n))
    dev.off()
  }, error=function(e) {
    message("Enrichment map failed: ", e$message)
    if(dev.cur()>1) dev.off()
  })
}
ego
}

ego_up_BP <- run_go(up_genes, universe, "BP", "Upregulated")
ego_up_CC <- run_go(up_genes, universe, "CC", "Upregulated")
ego_up_MF <- run_go(up_genes, universe, "MF", "Upregulated")

ego_down_BP <- run_go(down_genes, universe, "BP", "Downregulated")
ego_down_CC <- run_go(down_genes, universe, "CC", "Downregulated")
ego_down_MF <- run_go(down_genes, universe, "MF", "Downregulated")

make_composite_pdf <- function(ego_list, prefix, output_file) {
  plots <- lapply(names(ego_list), function(x) {
    ego <- ego_list[[x]]
    if(is.null(ego) || nrow(as.data.frame(ego))==0) return(NULL)
    dotplot(ego, showCategory=min(20, nrow(as.data.frame(ego)))) +
      ggtitle(paste0(prefix, " ", x))
  })
  plots <- Filter(Negate(is.null), plots)
  if(!length(plots)) return(NULL)

  combined <- wrap_plots(plots, ncol=length(plots))
  emap <- NULL
  bp <- ego_list[["BP"]]

  if(!is.null(bp) && nrow(as.data.frame(bp))>=2) {
    emap <- tryCatch(
      emapplot(pairwise_termsim(bp), showCategory=min(20, nrow(as.data.frame(bp)))),
      error=function(e) {message("BP enrichment map failed: ", e$message); NULL})
  }

  pdf(output_file, width=16, height=12)
  if(!is.null(emap)) print(combined/emap+plot_layout(heights=c(1, 1.2)))
  else print(combined)
  dev.off()
}

```

```

}

make_composite_pdf(
  list(BP=ego_up_BP,CC=ego_up_CC,MF=ego_up_MF),
  "Upregulated",
  file.path(output_dir,"Upregulated_GO_composite.pdf"))

make_composite_pdf(
  list(BP=ego_down_BP,CC=ego_down_CC,MF=ego_down_MF),
  "Downregulated",
  file.path(output_dir,"Downregulated_GO_composite.pdf"))

cat("\nGO ANALYSIS COMPLETE\nOutput directory: ",output_dir,"\n",sep="")

```

```

#!/bin/bash
#SBATCH --account=ibms
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --job-name="gsea"
#SBATCH --mail-user=[REDACTED]
#SBATCH --mail-type=ALL

#Load Conda module
module load python/miniconda3-py3.9

#Initialize Conda
source "$(conda info --base)/etc/profile.d/conda.sh"

#Activate environment
conda activate rnaseq-norm-env

#Install fgsea
conda install -y -c conda-forge bioconductor-fgsea

#Install inside R
Rscript - <<'EOF'
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager", repos="https://cloud.r-project.org")

BiocManager::install(c("clusterProfiler", "DOSE", "enrichplot",
  "ReactomePA", "org.Hs.eg.db",
  "GSEABase", "AnnotationDbi", "biomaRt",
  "msigdb"))

EOF

#Run script
Rscript /scratch/[REDACTED]/gsea.r

```

[gsea.r](#)

```

#!/usr/bin/env Rscript
suppressPackageStartupMessages({
  library(data.table); library(fgsea); library(ggplot2)
  library(gridExtra); library(pheatmap)
})

# Paths
dge_file<-"/scratch/[REDACTED]/GSEA/DGE_results_R_vs_NR.csv"
gmt_file<-"/scratch/[REDACTED]/GSEA/gene_sets.gmt"
outdir<-"/scratch/[REDACTED]/GSEA"
plot_dir<-file.path(outdir,"Enrichment_Plots")

```

```

heat_dir<-file.path(outdir,"LeadingEdge_Heatmaps")
dir.create(plot_dir,showWarnings=FALSE,recursive=TRUE)
dir.create(heat_dir,showWarnings=FALSE,recursive=TRUE)

# DESeq2 results and Wald-statistic ranking
res<-fread(dge_file)
stopifnot(all(c("GeneID","log2FoldChange","lfcSE","padj")%in%names(res)))
res[,stat:=log2FoldChange/lfcSE]
res<-res[is.finite(stat)&!is.na(GeneID)&GeneID!=""]
res<-res[!duplicated(GeneID)]
ranks<-res$stat; names(ranks)<-res$GeneID; ranks<-sort(ranks,decreasing=TRUE)
cat("Ranking vector length:",length(ranks),"\n")

# FGSEA
pathways<-gmtPathways(gmt_file)
cat("Loaded",length(pathways),"pathways\n")
fgseaRes<-fgsea(pathways=pathways,stats=ranks,minSize=15,maxSize=500,eps=0)
setorder(fgseaRes,padj)
fwrite(fgseaRes,file.path(outdir,"fgsea_results_DESeq2_stat.csv"))
fgseaSig<-fgseaRes[padj<.05]
if(!nrow(fgseaSig))stop("No significant pathways (padj < 0.05). Pipeline
stopped.")

# Enrichment plots and leading-edge heatmaps
safe_name<-function(x)gsub("[^A-Za-z0-9_\\-]", "_",x)
plot_index<-list()

for(i in seq_len(nrow(fgseaSig))){
  pw<-fgseaSig$pathway[i]; NES<-fgseaSig$NES[i]; padj<-fgseaSig$padj[i]
  le<-fgseaSig$leadingEdge[[i]]; direction<-
ifelse(NES>0,"Upregulated","Downregulated")
  subdir<-file.path(plot_dir,direction);
  dir.create(subdir,showWarnings=FALSE,recursive=TRUE)
  tick_pos<-match(le,names(ranks)); tick_pos<-tick_pos[!is.na(tick_pos)]

  p<-plotEnrichment(pathways[[pw]],ranks)+
  geom_vline(xintercept=tick_pos,color="red",alpha=.15)+
  labs(title=pw,subtitle=paste0("NES = ",round(NES,2)," | padj =
",signif(padj,3),
  "\nTop leading-edge genes: ",paste(head(le,5),collapse=" ")),
  x="Rank in ordered gene list (DESeq2 Wald statistic)",y="Running
enrichment score")+
  theme_minimal(base_size=10)+

  theme(plot.title=element_text(face="bold",size=11),plot.subtitle=element_text(si
ze=9))

  plot_file<-file.path(subdir,paste0("Enrichment_",safe_name(pw),".pdf"))
  ggsave(plot_file,p,width=6.5,height=5)

  le_df<-res[GeneID%in%le][order(-abs(stat)) ]
  if(nrow(le_df)>1){
    le_mat<-as.matrix(le_df$stat,drop=FALSE); rownames(le_mat)<-le_df$GeneID
    colnames(le_mat)<- "DESeq2_stat"
    heat_file<-file.path(heat_dir,paste0("Heatmap_",safe_name(pw),".pdf"))
    pdf(heat_file,width=4,height=max(4,nrow(le_mat)*.15))
    pheatmap(le_mat,cluster_rows=TRUE,cluster_cols=FALSE,
      main=paste0(pw," (Leading-edge genes)",fontsize_row=7)
    dev.off()
  }else heat_file<-NA

  plot_index[[i]]<-data.table(pathway=pw,NES=NES,padj=padj,direction=direction,
    enrichment_plot=plot_file,heatmap=heat_file)
}

```

```

# Significant pathway index
index_df<-rbindlist(plot_index)
pdf(file.path(outdir, "FGSEA_Significant_Pathway_Index.pdf"), width=8, height=10)
grid.table(index_df[, .(Pathway=pathway, NES=round(NES, 2), padj=signif(padj, 3), Direction=direction)])
dev.off()

# Bar plot: all significant pathways
top_bar<-copy(fgseaSig)
top_bar[, log10padj:=-log10(padj)]
top_bar[, padj_label:=paste0("padj = ", signif(padj, 3))]
p_bar<-ggplot(top_bar, aes(reorder(pathway, NES), NES, fill=log10padj)) +
  geom_bar(stat="identity") +
  geom_text(aes(label=padj_label, y=ifelse(NES>0, NES+.1, NES-.1)), size=3) +
  geom_hline(yintercept=0, linetype="dashed", color="grey40") +
  coord_flip() +
  labs(x="Pathway", y="Normalised Enrichment Score (NES)", fill="-log10(padj)",
        title="FGSEA (DESeq2 stat-ranked genes)\nSignificant pathways (padj <
0.05)") +
  scale_fill_gradient(low="lightblue", high="darkblue") +
  theme_minimal(base_size=9)
ggsave(file.path(outdir, "Top_Pathways_Barplot_DESeq2_padj_lt_0.05.pdf"),
        p_bar, width=8, height=6)

# Summary table
fwrite(fgseaRes[, .(pathway, NES, ES, padj, pval, size, leadingEdge)],
        file.path(outdir, "fgsea_summary_table_DESeq2.csv"))
cat("FGSEA pipeline complete.\nSignificant pathways:", nrow(fgseaSig), "\n")

```
